## Reflexive relation definition

A relation $R$ on a set $A$ is called **reflexive** means $(a, a) \in R$ for every element $a \in A$.

## Reflexive relation informally

*Informally*, every element is related to itself.

*Graphically*, there are self-loops (edge from a node back to itself) at every node.

## Symmetric relation definition

A relation $R$ on a set $A$ is called **symmetric** means $(b, a) \in R$ whenever $(a, b) \in R$, for all $a, b \in A$.

## Symmetric relation informally

*Informally*, order doesn't matter for this relation.

*Graphically*, every edge has a paired "backwards" edge so we might as well drop the arrows and think of edges as undirected.

## Transitive relation definition

A relation $R$ on a set $A$ is called **transitive** means whenever $(a, b) \in R$ and $(b, c) \in R$, then $(a, c) \in R$, for all $a, b, c \in A$.

## Transitive relation informally

*Informally*, chains of relations collapse.

*Graphically*, there's a shortcut between any endpoints of a chain of edges.

CC BY-NC-SA 2.0 Version November 6, 2026 (1)

## Antisymmetric relation definition

A relation $R$ on a set $A$ is called **antisymmetric** means $\forall a \in A \, \forall b \in A \; ( \; ( \; (a, b) \in R \wedge (b, a) \in R \; ) \rightarrow a = b \; )$

## Antisymmetric relation informally

*Informally*, the relation has directionality.

*Graphically*, can organize the nodes of the graph so that all non-self loop edges go up.

## Binary relation properties examples

When the domain is $\{a, b, c, d, e, f, g, h\}$ define a relation that is **not reflexive** and is **not symmetric** and is **not transitive**.

When the domain is $\{a, b, c, d, e, f, g, h\}$ define a relation that is **not reflexive** but is **symmetric** and is **transitive**.

When the domain is $\{a, b, c, d, e, f, g, h\}$ define a relation that is **symmetric** and is **antisymmetric**.

Is the relation $EQ_{\mathbb{R}}$ reflexive? symmetric? transitive? antisymmetric?

Is the relation $R_{(\mathbf{mod}\ 4)}$ reflexive? symmetric? transitive? antisymmetric?

## Equivalence relation definition

A relation is an **equivalence relation** means it is reflexive, symmetric, and transitive.

## Equivalence relation examples

*Example*: For $A = \mathcal{P}(\mathbb{R})$, the relation $EQ_{\mathbb{R}}$ defined on $A$ as

$$\{(X_1, X_2) \in \mathcal{P}(\mathbb{R}) \times \mathcal{P}(\mathbb{R}) \mid |X_1| = |X_2|\}$$

is an equivalence relation.

*Example*: For $A = \mathbb{Z}$, for each positive integer $n$, the relation $R_{(\mathbf{mod}\ n)}$ defined as the set of ordered pairs $(a, b)$ such that ($a$ **mod** $n = b$ **mod** $n$) is an equivalence relation.

CC BY-NC-SA 2.0 Version November 6, 2026 (2)

# Partial order definition

A relation is a **partial ordering** (or partial order) means it is reflexive, antisymmetric, and transitive.

# Hasse diagram definition

For a partial ordering, its **Hasse diagram** is a graph representing the relationship between elements in the ordering. The nodes (vertices) of the graph are the elements of the domain of the binary relation. The edges do not have arrow heads. The directionality of the partial order is indicated by the arrangements of the nodes. The nodes are arranged so that nodes connected to nodes above them by edges indicate that the relation holds between the lower node and the higher node. Moreover, the diagram omits self-loops and omits edges that are guaranteed by transitivity.

# Hasse diagram example

Draw the Hasse diagram of the partial order on the set $\{a, b, c, d, e, f, g\}$ defined as

$$\{(a,a),(b,b),(c,c),(d,d),(e,e),(f,f),(g,g),\\ (a,c),(a,d),(d,g),(a,g),(b,f),(b,e),(e,g),(b,g)\}$$

# Partition definition

A **partition** of a set $A$ is a set of non-empty, disjoint subsets $A_1, A_2, \cdots, A_n$ such that

$$A = \bigcup_{i=1}^{n} A_i = \{x \mid \exists i(x \in A_i)\}$$

CC BY-NC-SA 2.0 Version November 6, 2026 (3)

# Equivalence class definition

An **equivalence class** of an element $a \in A$ with respect to an equivalence relation $R$ on the set $A$ is the set

$$\{s \in A \mid (a, s) \in R\}$$

We write $[a]_R$ for this set, which is the equivalence class of $a$ with respect to $R$.

# Congruence classes mod four

*Recall*: We say $a$ is **congruent to** $b$ **mod** $n$ means $(a, b) \in R_{(\mathbf{mod}\ n)}$. A common notation is to write this as $a \equiv b (\mathbf{mod}\ n)$.

We can partition the set of integers using equivalence classes of $R_{(\mathbf{mod}\ 4)}$

$$\begin{aligned}
[0]_{R_{(\mathbf{mod}\ 4)}} &= \\
[1]_{R_{(\mathbf{mod}\ 4)}} &= \\
[2]_{R_{(\mathbf{mod}\ 4)}} &= \\
[3]_{R_{(\mathbf{mod}\ 4)}} &= \\
[4]_{R_{(\mathbf{mod}\ 4)}} &= \\
[5]_{R_{(\mathbf{mod}\ 4)}} &= \\
[-1]_{R_{(\mathbf{mod}\ 4)}} &=
\end{aligned}$$

$$\mathbb{Z} = [0]_{R_{(\mathbf{mod}\ 4)}} \cup [1]_{R_{(\mathbf{mod}\ 4)}} \cup [2]_{R_{(\mathbf{mod}\ 4)}} \cup [3]_{R_{(\mathbf{mod}\ 4)}}$$

# Modular arithmetic motivation

Integers are useful because they can be used to encode other objects and have multiple representations. However, infinite sets are sometimes expensive to work with computationally. Reducing our attention to a *partition of the integers* based on congrunce mod $n$, where each part is represented by a (not too large) integer gives a useful compromise where many algebraic properties of the integers are preserved, and we also get the benefits of a finite domain. Moreover, modular arithmetic is well-suited to model any cyclic behavior.

CC BY-NC-SA 2.0 Version November 6, 2026 (4)

# Equivalence relations partitions

**Claim**: For each $a \in U$, $[a]_E \neq \emptyset$.

**Proof**: Towards a _____ consider an arbitrary element $a$ in $U$. We will work to show that $[a]_E \neq \emptyset$, namely that $\exists x \in [a]_E$. By definition of equivalence classes, we can rewrite this goal as

$$\exists x \in U \ (\ (a, x) \in E\ )$$

Towards a _____, consider $x = a$, an element of $U$ by definition. By _____ of $E$, we know that $(a, a) \in E$ and thus the existential quantification has been proved.

**Claim**: For each $a \in U$, there is some $b \in U$ such that $a \in [b]_E$.

Towards a _____ consider an arbitrary element $a$ in $U$. By definition of equivalence classes, we can rewrite the goal as

$$\exists b \in U \ (\ (b, a) \in E\ )$$

Towards a _____, consider $b = a$, an element of $U$ by definition. By _____ of $E$, we know that $(a, a) \in E$ and thus the existential quantification has been proved.

**Claim**: For each $a, b \in U$ , ( $(a, b) \in E \ \rightarrow \ [a]_E = [b]_E$ ) and ( $(a, b) \notin E \ \rightarrow \ [a]_E \cap [b]_E = \emptyset$ )

**Corollary**: Given an equivalence relation $E$ on set $U$, $\{[x]_E \mid x \in U\}$ is a partition of $U$.

CC BY-NC-SA 2.0 Version November 6, 2026 (5)

# Modular arithmetic

**Modular arithmetic**:

**Informally**: can bring mod "inside" and do it first, for addition and for multiplication.

**Claim**: For $a, b, c, d \in \mathbb{Z}$ and positive integer $n$, if $a \equiv b$ ( **mod** $n$) and $c \equiv d$ ( **mod** $n$) then $a + c \equiv b + d$ ( **mod** $n$) and $ac \equiv bd$ ( **mod** $n$).

$123456 \times 789012$ **mod** $1000 =$

Smaller examples work too:

$(102 + 48)$ **mod** $10 =$

$(7 \cdot 10)$ **mod** $5 =$

Let's consider exponentiation:

$(2^5)$ **mod** $3 =$

To prove the claim: the following lemma is useful.

# Congruence mod n lemma

**Lemma**: For $a, b \in \mathbb{Z}$ and positive integer $n$, $(a, b) \in R_{(\textbf{mod } n)}$ if and only if $n|a - b$.

**Proof**:

CC BY-NC-SA 2.0 Version November 6, 2026 (6)

# Modular arithmetic final review

Compute the last digit of

$$(42)^{2026}$$

*Extra* Describe the pattern that helps you perform this computation and prove it using mathematical induction and/or modular arithmetic.

# Cardinality caution

*Caution*: we use familiar symbols to define cardinality, like $|\quad| \leq |\quad|$ and $|\quad| \geq |\quad|$ and $|\quad| = |\quad|$, but the meaning of these symbols depends on context. We've seen that vertical lines can mean absolute value (for real numbers), divisibility (for integers), and now sizes (for sets).

Now we see that $\leq$ and $\geq$ can mean comparing numbers or comparing sizes of sets. When the sets being compared are finite, the definitions of $|A| \leq |B|$ agree.

But, properties of numbers cannot be assumed when comparing cardinalities of infinite sets.

In a nutshell: cardinality of sets is defined via functions. This definition agrees with the usual notion of "size" for finite sets.

# Cardinality properties

### Properties of cardinality

$$\forall A\ (\ |A| = |A|\ )$$
$$\forall A\ \forall B\ (\ |A| = |B|\ \rightarrow\ |B| = |A|\ )$$
$$\forall A\ \forall B\ \forall C\ (\ (|A| = |B|\ \wedge\ |B| = |C|)\ \rightarrow\ |A| = |C|\ )$$

*Extra practice with proofs:* Use the definitions of bijections to prove these properties.

CC BY-NC-SA 2.0 Version November 6, 2026 (7)

# Cantor schroder bernstein theorem

**Cantor-Schroder-Bernstein Theorem**: For all nonempty sets,

$$|A| = |B| \quad \text{if and only if} \quad (|A| \leq |B| \text{ and } |B| \leq |A|) \quad \text{if and only if} \quad (|A| \geq |B| \text{ and } |B| \geq |A|)$$

To prove $|A| = |B|$, we can do any **one** of the following

- Prove there exists a bijection $f : A \to B$;
- Prove there exists a bijection $f : B \to A$;
- Prove there exists two functions $f_1 : A \to B,\ f_2 : B \to A$ where each of $f_1, f_2$ is one-to-one.
- Prove there exists two functions $f_1 : A \to B,\ f_2 : B \to A$ where each of $f_1, f_2$ is onto.

# Countably infinite definition

**Definition**: A set $A$ is **countably infinite** means it is the same size as $\mathbb{N}$.

CC BY-NC-SA 2.0 Version November 6, 2026 (8)

# Countably infinite examples sets of numbers

**Natural numbers** $\mathbb{N}$ *List*: 0 1 2 3 4 5 6 7 8 9 10 . . .

$identity : \mathbb{N} \to \mathbb{N}$ with $identity(n) = n$

*Claim*: *identity* is a bijection. *Proof*: Ex. **Corollary**: $|\mathbb{N}| = |\mathbb{N}|$

**Positive integers** $\mathbb{Z}^+$ *List*: 1 2 3 4 5 6 7 8 9 10 11 . . .

$positives : \mathbb{N} \to \mathbb{Z}^+$ with $positives(n) = n + 1$

*Claim*: *positives* is a bijection. *Proof*: Ex. **Corollary**: $|\mathbb{N}| = |\mathbb{Z}^+|$

**Negative integers** $\mathbb{Z}^-$ *List*: $-1$ $-2$ $-3$ $-4$ $-5$ $-6$ $-7$ $-8$ $-9$ $-10$ $-11$. . .

$negatives : \mathbb{N} \to \mathbb{Z}^-$ with $negatives(n) = -n - 1$

*Claim*: *negatives* is a bijection. **Corollary**: $|\mathbb{N}| = |\mathbb{Z}^-|$

*Proof*: We need to show it is a well-defined function that is one-to-one and onto.

- Well-defined?

  Consider an arbitrary element of the domain, $n \in \mathbb{N}$. We need to show it maps to exactly one element of $\mathbb{Z}^-$.

- One-to-one?

  Consider arbitrary elements of the domain $a, b \in \mathbb{N}$. We need to show that

  $$( \ negatives(a) = negatives(b) \ ) \to (a = b)$$

- Onto?

  Consider arbitrary element of the codomain $b \in \mathbb{Z}^-$. We need witness in $\mathbb{N}$ that maps to $b$.

**Integers** $\mathbb{Z}$ *List*: 0 $-1$ 1 $-2$ 2 $-3$ 3 $-4$ 4 $-5$ 5. . .

$$f : \mathbb{Z} \to \mathbb{N} \text{ with } f(x) = \begin{cases} 2x & \text{if } x \geq 0 \\ -2x - 1 & \text{if } x < 0 \end{cases}$$

*Claim*: $f$ is a bijection. *Proof*: Ex. **Corollary**: $|\mathbb{Z}| = |\mathbb{N}|$

CC BY-NC-SA 2.0 Version November 6, 2026 (9)

# Countably infinite examples other sets

### More examples of countably infinite sets

**Claim**: $S$ is countably infinite

*Similarly: The set of all strings over a specific alphabet is countably infinite.*

Bijection using alphabetical-ish ordering (first order by length, then alphabetically among strings of same length) of strands

**Claim**: $L$ is countably infinite

$$list : \mathbb{N} \to L$$
$$list(n) = (n, [])$$

$$toNum : L \to \mathbb{N}$$
$$toNum([]) = 0$$
$$toNum(\ (n, l)\ ) = 2^n 3^{toNum(l)} \qquad \text{for } n \in \mathbb{N},\ l \in L$$

**Claim**: $|\mathbb{Z}^+| = |\mathbb{Q}|$

One-to-one function from $\mathbb{Z}^+$ to $\mathbb{Q}$ is $f_1 : \mathbb{Z}^+ \to \mathbb{Q}$ with $f_1(n) = n$ for all $n \in \mathbb{N}$.

$$f_2 : \mathbb{Q} \to \mathbb{Z} \times \mathbb{Z}$$
$$f_2(x) = \begin{cases} (0,1) & \text{if } x = 0 \\ (p,q) & \text{if } x = \frac{p}{q}, \\ & gcd(p,q) = 1,\ q > 0 \end{cases}$$

$$f_3 : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}^+ \times \mathbb{Z}^+$$
$$f_3(\ (x,y)\ ) = \begin{cases} (2x+2, 2y+2) & \text{if } x \geq 0, y \geq 0 \\ (-2x-1, 2y+2) & \text{if } x < 0, y \geq 0 \\ (2x+2, -2y+1) & \text{if } x \geq 0, y < 0 \\ (-2x-1, -2y-1) & \text{if } x < 0, y < 0 \end{cases}$$

$$f_4 : \mathbb{Z}^+ \times \mathbb{Z}^+ \to \mathbb{Z}^+$$
$$f_4(\ (x,y)\ ) = 2^x 3^y \qquad \text{for } x, y \in \mathbb{Z}^+$$

CC BY-NC-SA 2.0 Version November 6, 2026 (10)

# Cardinality categories

A set $A$ is **finite** means it is empty or it is the same size as $\{1, \ldots, n\}$ for some $n \in \mathbb{N}$.

A set $A$ is **countably infinite** means it is the same size as $\mathbb{N}$. *Notice: all countably infinite sets are the same size as each other.*

A set $A$ is **countable** means it is either finite or countably infinite.

A set $A$ is **uncountable** means it is not countable.

CC BY-NC-SA 2.0 Version November 6, 2026 (11)

# Cardinality countability lemmas

### Lemmas about countable and uncountable sets

**Lemma**: If $A$ is a subset of a countable set, then it's countable.

**Lemma**: If $A$ is a superset of an uncountable set, then it's uncountable.

**Lemma**: If $A$ and $B$ are countable sets, then $A \cup B$ is countable and $A \cap B$ is countable.

**Lemma**: If $A$ and $B$ are countable sets, then $A \times B$ is countable.

*Generalize pairing ideas from* $\mathbb{Z}^+ \times \mathbb{Z}^+$ *to* $\mathbb{Z}^+$

**Lemma**: If $A$ is a subset of $B$ , to show that $|A| = |B|$, it's enough to give one-to-one function from $B$ to $A$ or an onto function from $A$ to $B$.

CC BY-NC-SA 2.0 Version November 6, 2026 (12)

# Cardinality power sets

*Recall*: When $U$ is a set, $\mathcal{P}(U) = \{X \mid X \subseteq U\}$

*Key idea*: For finite sets, the power set of a set has strictly greater size than the set itself. Does this extend to infinite sets?

**Definition**: For two sets $A, B$, we use the notation $|A| < |B|$ to denote $(\ |A| \leq |B|\ ) \wedge \neg(\ |A| = |B|)$.

| | | |
|---|---|---|
| $\emptyset = \{\}$ | $\mathcal{P}(\emptyset) = \{\emptyset\}$ | $\lvert\emptyset\rvert < \lvert\mathcal{P}(\emptyset)\rvert$ |
| $\{1\}$ | $\mathcal{P}(\{1\}) = \{\emptyset, \{1\}\}$ | $\lvert\{1\}\rvert < \lvert\mathcal{P}(\{1\})\rvert$ |
| $\{1, 2\}$ | $\mathcal{P}(\{1, 2\}) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ | $\lvert\{1, 2\}\rvert < \lvert\mathcal{P}(\{1, 2\})\rvert$ |

### $\mathbb{N}$ and its power set

Example elements of $\mathbb{N}$

Example elements of $\mathcal{P}(\mathbb{N})$

**Claim**: $|\mathbb{N}| \leq |\mathcal{P}(\mathbb{N})|$

CC BY-NC-SA 2.0 Version November 6, 2026 (13)

**Claim**: There is an uncountable set. Example: _____

**Proof**: By definition of countable, since _____ is not finite, **to show** is $|\mathbb{N}| \neq |\mathcal{P}(\mathbb{N})|$ .

Rewriting using the definition of cardinality, **to show** is

Towards a proof by universal generalization, consider an arbitrary function $f : \mathbb{N} \to \mathcal{P}(\mathbb{N})$.

**To show**: $f$ is not a bijection. It's enough to show that $f$ is not onto.

Rewriting using the definition of onto, **to show**:

$$\neg\forall B \in \mathcal{P}(\mathbb{N})\ \exists a \in \mathbb{N}\ (\ f(a) = B\ )$$

. By logical equivalence, we can write this as an existential statement:

---

In search of a witness, define the following collection of nonnegative integers:

$$D_f = \{n \in \mathbb{N}\ \mid\ n \notin f(n)\}$$

. By definition of power set, since all elements of $D_f$ are in $\mathbb{N}$, $D_f \in \mathcal{P}(\mathbb{N})$. It's enough to prove the following Lemma:

**Lemma**: $\forall a \in \mathbb{N}\ (\ f(a) \neq D_f\ )$.

**Proof of lemma**:

By the Lemma, we have proved that $f$ is not onto, and since $f$ was arbitrary, there are no onto functions from $\mathbb{N}$ to $\mathcal{P}(\mathbb{N})$. QED

**Where does $D_f$ come from?** The idea is to build a set that would "disagree" with each of the images of $f$ about some element.

| $n \in \mathbb{N}$ | $f(n) = X_n$ | Is $0 \in X_n$? | Is $1 \in X_n$? | Is $2 \in X_n$? | Is $3 \in X_n$? | Is $4 \in X_n$? | ... | Is $n \in D_f$? |
|---|---|---|---|---|---|---|---|---|
| 0 | $f(0) = X_0$ | **Y / N** | Y / N | Y / N | Y / N | Y / N | ... | **N / Y** |
| 1 | $f(1) = X_1$ | Y / N | **Y / N** | Y / N | Y / N | Y / N | ... | **N / Y** |
| 2 | $f(2) = X_2$ | Y / N | Y / N | **Y / N** | Y / N | Y / N | ... | **N / Y** |
| 3 | $f(3) = X_3$ | Y / N | Y / N | Y / N | **Y / N** | Y / N | ... | **N / Y** |
| 4 | $f(4) = X_4$ | Y / N | Y / N | Y / N | Y / N | **Y / N** | ... | **N / Y** |
| $\vdots$ | | | | | | | | |

CC BY-NC-SA 2.0 Version November 6, 2026 (14)

# Cardinality rationals reals

### Comparing $\mathbb{Q}$ and $\mathbb{R}$

Both $\mathbb{Q}$ and $\mathbb{R}$ have no greatest element.

Both $\mathbb{Q}$ and $\mathbb{R}$ have no least element.

The quantified statement

$$\forall x \forall y (x < y \rightarrow \exists z (x < z < y))$$

is true about both $\mathbb{Q}$ and $\mathbb{R}$.

Both $\mathbb{Q}$ and $\mathbb{R}$ are infinite. But, $\mathbb{Q}$ is countably infinite whereas $\mathbb{R}$ is uncountable.

### The set of real numbers

$\mathbb{Z} \subsetneq \mathbb{Q} \subsetneq \mathbb{R}$

**Order axioms** (Rosen Appendix 1):

| | |
|---|---|
| Reflexivity | $\forall a \in \mathbb{R} (a \leq a)$ |
| Antisymmetry | $\forall a \in \mathbb{R}\ \forall b \in \mathbb{R}\ (\ (a \leq b\ \wedge\ b \leq a) \rightarrow (a = b)\ )$ |
| Transitivity | $\forall a \in \mathbb{R}\ \forall b \in \mathbb{R}\ \forall c \in \mathbb{R}\ (\ (a \leq b \wedge b \leq c)\ \rightarrow\ (a \leq c)\ )$ |
| Trichotomy | $\forall a \in \mathbb{R}\ \forall b \in \mathbb{R}\ (\ (a = b\ \vee\ b > a\ \vee\ a < b)$ |

**Completeness axioms** (Rosen Appendix 1):

| | |
|---|---|
| Least upper bound | Every nonempty set of real numbers that is bounded above has a least upper bound |
| Nested intervals | For each sequence of intervals $[a_n, b_n]$ where, for each $n$, $a_n < a_{n+1} < b_{n+1} < b_n$, there is at least one real number $x$ such that, for all $n$, $a_n \leq x \leq b_n$. |

Each real number $r \in \mathbb{R}$ is described by a function to give better and better approximations

$$x_r : \mathbb{Z}^+ \rightarrow \{0, 1\} \qquad \text{where } x_r(n) = n^{th} \text{ bit in binary expansion of } r$$

| $r$ | Binary expansion | $x_r$ |
|---|---|---|
| 0.1 | $0.00011001\ldots$ | $x_{0.1}(n) = \begin{cases} 0 & \text{if } n > 1 \text{ and } (n \textbf{ mod } 4) = 2 \\ 0 & \text{if } n = 1 \text{ or if } n > 1 \text{ and } (n \textbf{ mod } 4) = 3 \\ 1 & \text{if } n > 1 \text{ and } (n \textbf{ mod } 4) = 0 \\ 1 & \text{if } n > 1 \text{ and } (n \textbf{ mod } 4) = 1 \end{cases}$ |
| $\sqrt{2} - 1 = 0.4142135\ldots$ | $0.01101010\ldots$ | Use linear approximations (tangent lines from calculus) to get algorithm for bounding error of successive operations. Define $x_{\sqrt{2}-1}(n)$ to be $n^{th}$ bit in approximation that has error less than $2^{-(n+1)}$. |

CC BY-NC-SA 2.0 Version November 6, 2026 (15)

**Claim**: $\{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}$ is uncountable.

*Approach 1*: Mimic proof that $\mathcal{P}(\mathbb{Z}^+)$ is uncountable.

**Proof**: By definition of countable, since $\{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}$ is not finite, **to show** is $|\mathbb{N}| \neq |\{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}|$ .

**To show** is $\forall f : \mathbb{Z}^+ \to \{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}$ ($f$ is not a bijection) . Towards a proof by universal generalization, consider an arbitrary function $f : \mathbb{Z}^+ \to \{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}$. **To show**: $f$ is not a bijection. It's enough to show that $f$ is not onto. Rewriting using the definition of onto, **to show**:

$$\exists x \in \{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}\ \forall a \in \mathbb{N}\ (\ f(a) \neq x\ )$$

In search of a witness, define the following real number by defining its binary expansion

$$d_f = 0.b_1b_2b_3\cdots$$

where $b_i = 1 - b_{ii}$ where $b_{jk}$ is the coefficient of $2^{-k}$ in the binary expansion of $f(j)$. Since[1] $d_f \neq f(a)$ for any positive integer $a$, $f$ is not onto.

*Approach 2*: Nested closed interval property

**To show** $f : \mathbb{N} \to \{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}$ is not onto. **Strategy**: Build a sequence of nested closed intervals that each avoid some $f(n)$. Then the real number that is in all of the intervals can't be $f(n)$ for any $n$. Hence, $f$ is not onto.

Consider the function $f : \mathbb{N} \to \{r \in \mathbb{R} \mid 0 \leq r \wedge r \leq 1\}$ with $f(n) = \frac{1+\sin(n)}{2}$

| $n$ | $f(n)$ | Interval that avoids $f(n)$ |
|---|---|---|
| 0 | 0.5 | |
| 1 | 0.920735... | |
| 2 | 0.954649... | |
| 3 | 0.570560... | |
| 4 | 0.121599... | |
| ⋮ | | |

[1] There's a subtle imprecision in this part of the proof as presented, but it can be fixed.

CC BY-NC-SA 2.0 Version November 6, 2026 (16)

# Cardinality uncountable examples

- The power set of any countably infinite set is uncountable. For example:

  $$\mathcal{P}(\mathbb{N}), \mathcal{P}(\mathbb{Z}^+), \mathcal{P}(\mathbb{Z})$$

  are each uncountable.

- The closed interval $\{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$, any other nonempty closed interval of real numbers whose endpoints are unequal, as well as the related intervals that exclude one or both of the endpoints.

- The set of all real numbers $\mathbb{R}$ is uncountable and the set of irrational real numbers $\overline{\mathbb{Q}}$ is uncountable.

# Cardinality recap

The set of positive integers $\mathbb{Z}^+$ is countably infinite.

The set of integers $\mathbb{Z}$ is countably infinite and is a proper superset of $\mathbb{Z}^+$. In fact, the set difference

$$\mathbb{Z} \setminus \mathbb{Z}^+ = \{x \in \mathbb{Z} \mid x \notin \mathbb{Z}^+\} = \{x \in \mathbb{Z} \mid x \leq 0\}$$

is countably infinite.

The set of rationals $\mathbb{Q} = \left\{\frac{p}{q} \mid p \in \mathbb{Z} \text{ and } q \in \mathbb{Z} \text{ and } q \neq 0\right\}$ is countably infinite.

The set of real numbers $\mathbb{R}$ is uncountable. In fact, the closed interval $\{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$, any other nonempty closed interval of real numbers whose endpoints are unequal, as well as the related intervals that exclude one or both of the endpoints are each uncountable. The set of **irrational** numbers $\overline{\mathbb{Q}} = \mathbb{R} - \mathbb{Q} = \{x \in \mathbb{R} \mid x \notin \mathbb{Q}\}$ is uncountable.

We can classify any set as

- **Finite** size. Fact: For each positive number $n$, for any sets $X$ and $Y$ each size $n$, there is a bijection between $X$ and $Y$.

- **Countably Infinite**. Fact: for any countably infinite sets $X$ and $Y$, there is a bijection between $X$ and $Y$.

- **Uncountable**. Examples: $\mathcal{P}(\mathbb{N})$, the power set of any infinite set, the set of real numbers, any nonempty interval of real numbers. Fact: there are (many) examples of uncountable sets that do not have a bijection between them.

CC BY-NC-SA 2.0 Version November 6, 2026 (17)

# Cardinality final review

**True or False**? For all sets $A$ and $B$ where $A \subseteq B$, if $A$ is infinite then $B$ is finite.

**True or False**? For all sets $A$ and $B$ where $A \subseteq B$, if $A$ is countable then $B$ is countable.

**True or False**? For all sets $A$ and $B$ where $A \subseteq B$, if $B$ is infinite then $A$ is finite.

**True or False**? For all sets $A$ and $B$ where $A \subseteq B$, if $B$ is uncountable then $A$ is countable.

# Binary relation definition

**Definition**: When $A$ and $B$ are sets, we say any subset of $A \times B$ is a **binary relation**. A relation $R$ can also be represented as

- A function $f_{TF} : A \times B \to \{T, F\}$ where, for $a \in A$ and $b \in B$, $f_{TF}(\ (a, b)\ ) = \begin{cases} T & \text{when } (a, b) \in R \\ F & \text{when } (a, b) \notin R \end{cases}$
- A function $f_{\mathcal{P}} : A \to \mathcal{P}(B)$ where, for $a \in A$, $f_{\mathcal{P}}(a) = \{b \in B \mid (a, b) \in R\}$

When $A$ is a set, we say any subset of $A \times A$ is a (binary) **relation** on $A$.

# Relations as graphs

For relation $R$ on a set $A$, we can represent this relation as a **graph**: a collection of nodes (vertices) and edges (arrows). The nodes of the graph are the elements of $A$ and there is an edge from $a$ to $b$ exactly when $(a, b) \in R$.

# Binary relation examples

*Example*: For $A = \mathcal{P}(\mathbb{R})$, we can define the relation $EQ_{\mathbb{R}}$ on $A$ as

$$\{(X_1, X_2) \in \mathcal{P}(\mathbb{R}) \times \mathcal{P}(\mathbb{R}) \mid |X_1| = |X_2|\}$$

*Example*: Let $R_{(\textbf{mod } n)}$ be the set of all pairs of integers $(a, b)$ such that ($a$ **mod** $n = b$ **mod** $n$). Then $a$ is **congruent to** $b$ **mod** $n$ means $(a, b) \in R_{(\textbf{mod } n)}$. A common notation is to write this as $a \equiv b (\textbf{mod } n)$.

$R_{(\textbf{mod } n)}$ is a relation on the set _____

Some example elements of $R_{(\textbf{mod } 4)}$ are:

# Rna rnalen basecount definitions

*Recall the definitions*: The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

The function *rnalen* that computes the length of RNA strands in $S$ is defined recursively by:

| | | | |
|---|---|---|---|
| | | $rnalen : S$ | $\to \mathbb{Z}^+$ |
| Basis Step: | If $b \in B$ then | $rnalen(b)$ | $= 1$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then | $rnalen(sb)$ | $= 1 + rnalen(s)$ |

The function *basecount* that computes the number of a given base $b$ appearing in a RNA strand $s$ is defined recursively by:

| | | | | |
|---|---|---|---|---|
| | | $basecount : S \times B$ | $\to \mathbb{N}$ | |
| Basis Step: | If $b_1 \in B, b_2 \in B$ | $basecount(\ (b_1, b_2)\ )$ | $= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ | |
| Recursive Step: | If $s \in S, b_1 \in B, b_2 \in B$ | $basecount(\ (sb_1, b_2)\ )$ | $= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | |

# Quiz translating counting quantifiers

Suppose $P(x)$ is a predicate over a domain $D$.

1. To translate the statement "There are at least two elements in $D$ where the predicate $P$ evaluates to true", we could write

$$\exists x_1 \in D\, \exists x_2 \in D\, (P(x_1) \wedge P(x_2) \wedge _____)$$

2. To translate the statement "There are at most two elements in $D$ where the predicate $P$ evaluates to true", we could write

$$\forall x_1 \in D\, \forall x_2 \in D\, \forall x_3 \in D\ (\ (\ P(x_1) \wedge P(x_2) \wedge P(x_3)\ ) \to (\ x_1 = x_2 \vee x_2 = x_3 \vee x_1 = x_3\ )\ )$$

   Why?

CC BY-NC-SA 2.0 Version November 6, 2026 (19)

# Proof strategies quantification finite domain

When a predicate $P(x)$ is over a **finite** domain:

- To show that $\forall x P(x)$ is true: check that $P(x)$ evaluates to $T$ at each domain element by evaluating over and over. This is called "Proof of universal by **exhaustion**".
- To show that $\forall x P(x)$ is false: find a **counterexample**, a domain element where $P(x)$ evaluates to $F$.
- To show that $\exists x P(x)$ is true: find a **witness**, a domain element where $P(x)$ evaluates to $T$.
- To show that $\exists x P(x)$ is false: check that $P(x)$ evaluates to $F$ at each domain element by evaluating over and over. DeMorgan's Law gives that $\neg\exists x P(x) \quad \equiv \quad \forall x \neg P(x)$ so this amounts to a proof of universal by exhaustion.

# Proof strategy universal generalization

**New! Proof by universal generalization**: To prove that $\forall x\, P(x)$ is true, we can take an arbitrary element $e$ from the domain of quantification and show that $P(e)$ is true, without making any assumptions about $e$ other than that it comes from the domain.
An **arbitrary** element of a set or domain is a fixed but unknown element from that set.

# Proof strategies conditionals

**New! Proof of conditional by direct proof**: To prove that the conditional statement $p \to q$ is true, we can assume $p$ is true and use that assumption to show $q$ is true.

**New! Proof of conditional by contrapositive proof**: To prove that the implication $p \to q$ is true, we can assume $q$ is false and use that assumption to show $p$ is also false.

**New! Proof of disjuction using equivalent conditional**: To prove that the disjunction $p \vee q$ is true, we can rewrite it equivalently as $\neg p \to q$ and then use direct proof or contrapositive proof.

# Proof strategies proof by cases

**New! Proof by Cases**: To prove $q$, we can work by cases by first describing all possible cases we might be in and then showing that each one guarantees $q$. Formally, if we know that $p_1 \vee p_2$ is true, and we can show that $(p_1 \to q)$ is true and we can show that $(p_2 \to q)$, then we can conclude $q$ is true.

CC BY-NC-SA 2.0 Version November 6, 2026 (20)

# Proof strategies ands

**New! Proof of conjunctions with subgoals**: To show that $p \wedge q$ is true, we have two subgoals: subgoal (1) prove $p$ is true; and, subgoal (2) prove $q$ is true.

To show that $p \wedge q$ is false, it's enough to prove that $\neg p$.
To show that $p \wedge q$ is false, it's enough to prove that $\neg q$.

# Sets equality subset definition

**Definitions**:

A **set** is an unordered collection of elements. When $A$ and $B$ are sets, $A = B$ (set equality) means

$$\forall x(x \in A \leftrightarrow x \in B)$$

When $A$ and $B$ are sets, $A \subseteq B$ ("$A$ is a **subset** of $B$") means

$$\forall x(x \in A \rightarrow x \in B)$$

When $A$ and $B$ are sets, $A \subsetneq B$ ("$A$ is a **proper subset** of $B$") means

$$(A \subseteq B) \wedge (A \neq B)$$

# Sets proof strategies

To prove that one set is a subset of another, e.g. to show $A \subseteq B$:

To prove that two sets are equal, e.g. to show $A = B$:

CC BY-NC-SA 2.0 Version November 6, 2026 (21)

## Sets equality example

Example: $\{43, 7, 9\} = \{7, 43, 9, 7\}$

CC BY-NC-SA 2.0 Version November 6, 2026 (22)

# Sets basic proofs

**Prove** or **disprove**: $\{\texttt{A}, \texttt{C}, \texttt{U}, \texttt{G}\} \subseteq \{\texttt{AA}, \texttt{AC}, \texttt{AU}, \texttt{AG}\}$

**Prove** or **disprove**: For some set $Y$, $\emptyset \in Y$.

**Prove** or **disprove**: For every set $Y$, $\emptyset \in you$.

**Prove** or **disprove**: The empty set is a subset of every set.

CC BY-NC-SA 2.0 Version November 6, 2026 (23)

**Prove** or **disprove**: The empty set is a proper subset of every set.

**Prove** or **disprove**: $\{4, 6\} \subseteq \{n \mid \exists c \in \mathbb{Z}(n = 4c)\}$

**Prove** or **disprove**: $\{4, 6\} \subseteq \{n \textbf{ mod } 10 \mid \exists c \in \mathbb{Z}(n = 4c)\}$

CC BY-NC-SA 2.0 Version November 6, 2026 (24)

# Proofs signposting

Consider . . . , an **arbitrary** . . . . **Assume** . . . , we **want to show** that . . . . Which is what was needed, so the proof is complete □.

*or, in other words:*

Let . . . be an **arbitrary** . . . . **Assume** . . . , **WTS** that . . . **QED**.

CC BY-NC-SA 2.0 Version November 6, 2026 (25)

# Set operations union intersection powerset

**Cartesian product**: When $A$ and $B$ are sets,

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

Example: $\{43, 9\} \times \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} \times \emptyset =$

**Union**: When $A$ and $B$ are sets,

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

Example: $\{43, 9\} \cup \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} \cup \emptyset =$

**Intersection**: When $A$ and $B$ are sets,

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

Example: $\{43, 9\} \cap \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} \cap \emptyset =$

**Set difference**: When $A$ and $B$ are sets,

$$A - B = \{x \mid x \in A \wedge x \notin B\}$$

Example: $\{43, 9\} - \{9, \mathbb{Z}\} =$

Example: $\mathbb{Z} - \emptyset =$

**Disjoint sets**: sets $A$ and $B$ are disjoint means $A \cap B = \emptyset$

Example: $\{43, 9\}, \{9, \mathbb{Z}\}$ are not disjoint

Example: The sets $\mathbb{Z}$ and $\emptyset$ are disjoint

**Power set**: When $U$ is a set, $\mathcal{P}(U) = \{X \mid X \subseteq U\}$

Example: $\mathcal{P}(\{43, 9\}) =$

Example: $\mathcal{P}(\emptyset) =$

CC BY-NC-SA 2.0 Version November 6, 2026 (26)

## Sets basic proofs operations

Let $W = \mathcal{P}(\{1, 2, 3, 4, 5\})$

Example elements in $W$ are:

**Prove** or **disprove**: $\forall A \in W \, \forall B \in W \; (A \subseteq B \; \rightarrow \; \mathcal{P}(A) \subseteq \mathcal{P}(B))$

**Prove** or **disprove**: $\forall A \in W \, \forall B \in W \; (\mathcal{P}(A) = \mathcal{P}(B) \; \rightarrow \; A = B)$

**Prove** or **disprove**: $\forall A \in W \, \forall B \in W \, \forall C \in W \; (A \cup B = A \cup C \; \rightarrow \; B = C)$

## Proof strategies road map

We now have propositional and predicate logic that can help us express statements about any domain. We will develop proof strategies to craft valid argument for proving that such statements are true or disproving them (by showing they are false). We will practice these strategies with statements about sets and numbers, both because they are familiar and because they can be used to build cryptographic systems. Then we will apply proof strategies more broadly to prove statements about data structures and machine learning applications.

## Numbers facts

1. Addition and multiplication of real numbers are each commutative and associative.

2. The product of two positive numbers is positive, of two negative numbers is positive, and of a positive and a negative number is negative.

3. The sum of two integers, the product of two integers, and the difference between two integers are each integers.

4. For every integer $x$ there is no integer strictly between $x$ and $x + 1$,

5. When $x, y$ are positive integers, $xy \geq x$ and $xy \geq y$.

# Factoring definition

**Definition**: When $a$ and $b$ are integers and $a$ is nonzero, $a$ **divides** $b$ means there is an integer $c$ such that $b = ac$ .

Symbolically, $F(\ (a, b)\ ) =$ and is a predicate over the domain _____

Other (synonymous) ways to say that $F(\ (a, b)\ )$ is true:

$a$ is a **factor** of $b$ $a$ is a **divisor** of $b$ $b$ is a **multiple** of $a$ $a|b$

When $a$ is a positive integer and $b$ is any integer, $a|b$ exactly when $b$ **mod** $a = 0$

When $a$ is a positive integer and $b$ is any integer, $a|b$ exactly when $b = a \cdot (b$ **div** $a)$

# Factoring translation examples

*Translate these quantified statements by matching to English statement on right.*

| | |
|---|---|
| $\exists a \in \mathbb{Z}^{\neq 0}\ (\ F(\ (a, a)\ )\ )$ | Every nonzero integer is a factor of itself. |
| $\exists a \in \mathbb{Z}^{\neq 0}\ (\ \neg F(\ (a, a)\ )\ )$ | No nonzero integer is a factor of itself. |
| $\forall a \in \mathbb{Z}^{\neq 0}\ (\ F(\ (a, a)\ )\ )$ | At least one nonzero integer is a factor of itself. |
| $\forall a \in \mathbb{Z}^{\neq 0}\ (\ \neg F(\ (a, a)\ )\ )$ | Some nonzero integer is not a factor of itself. |

# Factoring basic claims

**Claim**: Every nonzero integer is a factor of itself.

**Proof**:

**Prove** or **Disprove**: There is a nonzero integer that does not divide its square.

**Prove** or **Disprove**: Every positive factor of a positive integer is less than or equal to it.

# Factoring basic claims continued

**Claim**: Every nonzero integer is a factor of itself and every nonzero integer divides its square.

CC BY-NC-SA 2.0 Version November 6, 2026 (28)

## Factoring even odd

**Definition**: an integer $n$ is **even** means that there is an integer $a$ such that $n = 2a$; an integer $n$ is **odd** means that there is an integer $a$ such that $n = 2a + 1$. Equivalently, an integer $n$ is **even** means $n \mod 2 = 0$; an integer $n$ is **odd** means $n \mod 2 = 1$. Also, an integer is even if and only if it is not odd.

## Prime number definition

**Definition**: An integer $p$ greater than 1 is called **prime** means the only positive factors of $p$ are 1 and $p$. A positive integer that is greater than 1 and is not prime is called composite.

CC BY-NC-SA 2.0 Version November 6, 2026 (29)

# Primes basic claims

*Extra examples*: Use the definition to prove that 1 is not prime, 2 is prime, 3 is prime, 4 is not prime, 5 is prime, 6 is not prime, and 7 is prime.

**True or False**: The statement "There are three consecutive positive integers that are prime."

*Hint*: These numbers would be of the form $p, p+1, p+2$ (where $p$ is a positive integer).

**Proof**: We need to show _____

**True or False**: The statement "There are three consecutive odd positive integers that are prime."

*Hint*: These numbers would be of the form $p, p+2, p+4$ (where $p$ is an odd positive integer).

**Proof**: We need to show _____

CC BY-NC-SA 2.0 Version November 6, 2026 (30)

## Logical operators binary truth table definition

| Input | | Output | | | | |
|---|---|---|---|---|---|---|
| | | Conjunction | Exclusive or | Disjunction | Conditional | Biconditional |
| $p$ | $q$ | $p \wedge q$ | $p \oplus q$ | $p \vee q$ | $p \rightarrow q$ | $p \leftrightarrow q$ |
| $T$ | $T$ | $T$ | $F$ | $T$ | $T$ | $T$ |
| $T$ | $F$ | $F$ | $T$ | $T$ | $F$ | $F$ |
| $F$ | $T$ | $F$ | $T$ | $T$ | $T$ | $F$ |
| $F$ | $F$ | $F$ | $F$ | $F$ | $T$ | $T$ |
| | | "$p$ and $q$" | "$p$ xor $q$" | "$p$ or $q$" | "if $p$ then $q$" | "$p$ if and only if $q$" |

## Hypothesis conclusion

The only way to make the conditional statement $p \rightarrow q$ false is to _____

The **hypothesis** of $p \rightarrow q$ is _____ The **antecedent** of $p \rightarrow q$ is _____

The **conclusion** of $p \rightarrow q$ is _____ The **consequent** of $p \rightarrow q$ is _____

## Converse inverse contrapositive

The **converse** of $p \rightarrow q$ is _____

The **inverse** of $p \rightarrow q$ is _____

The **contrapositive** of $p \rightarrow q$ is _____

CC BY-NC-SA 2.0 Version November 6, 2026 (31)

# Compound propositions recursive definition

We can use a recursive definition to describe all compound propositions that use propositional variables from a specified collection. Here's the definition for all compound propositions whose propositional variables are in $\{p, q\}$.

| | |
|---|---|
| Basis Step: | $p$ and $q$ are each a compound proposition |
| Recursive Step: | If $x$ is a compound proposition then so is $(\neg x)$ and if $x$ and $y$ are both compound propositions then so is each of $(x \wedge y), (x \oplus y), (x \vee y), (x \to y), (x \leftrightarrow y)$ |

# Compound propositions precedence

Order of operations (Precedence) for logical operators:

Negation, then conjunction / disjunction, then conditional / biconditionals.

Example: $\neg p \vee \neg q$ means $(\neg p) \vee (\neg q)$.

# Logical equivalence identities ififf

**(Some) logical equivalences**

$p \to q \equiv \neg p \vee q$

$p \to q \equiv \neg q \to \neg p$ **Contrapositive**

$\neg(p \to q) \equiv p \wedge \neg q$

$\neg(p \leftrightarrow q) \equiv p \oplus q$

$p \leftrightarrow q \equiv q \leftrightarrow p$

*Extra examples*:

$p \leftrightarrow q$ is not logically equivalent to $p \wedge q$ because _____

$p \to q$ is not logically equivalent to $q \to p$ because _____

CC BY-NC-SA 2.0 Version November 6, 2026 (32)

# Logical operators english synonyms

**Common ways to express logical operators in English**:

**Negation** $\neg p$ can be said in English as

- Not $p$.
- It's not the case that $p$.
- $p$ is false.

**Conjunction** $p \wedge q$ can be said in English as

- $p$ and $q$.
- Both $p$ and $q$ are true.
- $p$ but $q$.

**Exclusive or** $p \oplus q$ can be said in English as

- $p$ or $q$, but not both.
- Exactly one of $p$ and $q$ is true.

**Disjunction** $p \vee q$ can be said in English as

- $p$ or $q$, or both.
- $p$ or $q$ (inclusive).
- At least one of $p$ and $q$ is true.

**Conditional** $p \rightarrow q$ can be said in English as

- if $p$, then $q$.
- $p$ is sufficient for $q$.
- $q$ when $p$.
- $q$ whenever $p$.
- $p$ implies $q$.
- $q$ follows from $p$.
- $q$ is necessary for $p$.
- $p$ only if $q$.

**Biconditional**

- $p$ if and only if $q$.
- $p$ iff $q$.
- If $p$ then $q$, and conversely.
- $p$ is necessary and sufficient for $q$.

CC BY-NC-SA 2.0 Version November 6, 2026 (33)

## Compound propositions translation

**Translation**: Express each of the following sentences as compound propositions, using the given propositions.

"A sufficient condition for the warranty to be good is that you bought the computer less than a year ago"

$w$ is "the warranty is good"
$b$ is "you bought the computer less than a year ago"

"Whenever the message was sent from an unknown system, it is scanned for viruses."

$s$ is "The message is scanned for viruses"
$u$ is "The message was sent from an unknown system"

"I will complete my to-do list only if I put a reminder in my calendar"

$d$ is "I will complete my to-do list"
$c$ is "I put a reminder in my calendar"

## Consistency def

**Definition**: A collection of compound propositions is called **consistent** if there is an assignment of truth values to the propositional variables that makes each of the compound propositions true.

## Predicate definition

**Definition**: A **predicate** is a function from a given set (domain) to $\{T, F\}$.

A predicate can be applied, or **evaluated** at, an element of the domain.

Usually, a predicate *describes a property* that domain elements may or may not have.

Two predicates over the same domain are **equivalent** means they evaluate to the same truth values for all possible assignments of domain elements to the input. In other words, they are equivalent means that they are equal as functions.

To define a predicate, we must specify its domain and its value at each domain element. The rule assigning truth values to domain elements can be specified using a formula, English description, in a table (if the domain is finite), or recursively (if the domain is recursively defined).

Note: Since logical operations are functions, they can be part of the definitions of predicates.

# Predicate examples finite domain

| Input | Output | | |
|---|---|---|---|
| | $V(x)$ | $N(x)$ | $Mystery(x)$ |
| $x$ | $[x]_{2c,3} > 0$ | $[x]_{2c,3} < 0$ | |
| 000 | $F$ | | $T$ |
| 001 | $T$ | | $T$ |
| 010 | $T$ | | $T$ |
| 011 | $T$ | | $F$ |
| 100 | $F$ | | $F$ |
| 101 | $F$ | | $T$ |
| 110 | $F$ | | $F$ |
| 111 | $F$ | | $T$ |

The domain for each of the predicates $V(x)$, $N(x)$, $Mystery(x)$ is _____.

Fill in the table of values for the predicate $N(x)$ based on the formula given.

# Predicate truth set definition

**Definition**: The **truth set** of a predicate is the collection of all elements in its domain where the predicate evaluates to $T$.

Notice that specifying the domain and the truth set is sufficient for defining a predicate.

# Predicate truth set example

The truth set for the predicate $V(x)$ is _____.

The truth set for the predicate $N(x)$ is _____.

The truth set for the predicate $Mystery(x)$ is _____.

CC BY-NC-SA 2.0 Version November 6, 2026 (35)

# Quantification definition

The **universal quantification** of predicate $P(x)$ over domain $U$ is the statement "$P(x)$ for all values of $x$ in the domain $U$" and is written $\forall x P(x)$ or $\forall x \in U\ P(x)$. When the domain is finite, universal quantification over the domain is equivalent to iterated *conjunction* (ands).

The **existential quantification** of predicate $P(x)$ over domain $U$ is the statement "There exists an element $x$ in the domain $U$ such that $P(x)$" and is written $\exists x P(x)$ for $\exists x \in U\ P(x)$. When the domain is finite, existential quantification over the domain is equivalent to iterated *disjunction* (ors).

# Quantification logical equivalence

Statements involving predicates and quantifiers are **logically equivalent** means they have the same truth value no matter which predicates (domains and functions) are substituted in.

**Quantifier version of De Morgan's laws**: $\boxed{\neg\forall x P(x) \ \equiv\ \exists x\,(\neg P(x))}$ $\boxed{\neg\exists x Q(x) \ \equiv\ \forall x\,(\neg Q(x))}$

# Quantification witness counterexample

An element for which $P(x) = F$ is called a **counterexample** of $\forall x P(x)$. When there is a counterexample in the domain of predicate $P$, $\forall x P(x)$ is false.

An element for which $P(x) = T$ is called a **witness** of $\exists x P(x)$. When there is a witness in the domain of predicate $P$, then $\exists x P(x)$ is true.

CC BY-NC-SA 2.0 Version November 6, 2026 (36)

# Quantification examples finite domain

Examples of quantifications using $V(x), N(x), Mystery(x)$:

**True** or **False**: $\exists x\ (\ V(x) \vee N(x)\ )$

**True** or **False**: $\forall x\ (\ V(x) \vee N(x)\ )$

**True** or **False**: $\exists x\ (\ \neg V(x) \wedge \neg N(x)\ )$

**True** or **False**: $\forall x\ (\ V(x) \rightarrow N(x)\ )$

**True** or **False**: $\exists x\ (\ N(x) \leftrightarrow Mystery(x)\ )$

Rewrite $\neg\forall x\ (\ V(x) \oplus Mystery(x)\ )$ into a logically equivalent statement.

Notice that these are examples where the predicates have *finite* domain. How would we evaluate quantifications where the domain may be infinite?

# Rna rnalen basecount definitions

*Recall the definitions*: The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

The function *rnalen* that computes the length of RNA strands in $S$ is defined recursively by:

| | | | |
|---|---|---|---|
| | | $rnalen : S$ | $\rightarrow \mathbb{Z}^+$ |
| Basis Step: | If $b \in B$ then | $rnalen(b)$ | $= 1$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then | $rnalen(sb)$ | $= 1 + rnalen(s)$ |

The function *basecount* that computes the number of a given base $b$ appearing in a RNA strand $s$ is defined recursively by:

| | | | | |
|---|---|---|---|---|
| | | $basecount : S \times B$ | $\rightarrow \mathbb{N}$ | |
| Basis Step: | If $b_1 \in B, b_2 \in B$ | $basecount(\ (b_1, b_2)\ )$ | $= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ | |
| Recursive Step: | If $s \in S, b_1 \in B, b_2 \in B$ | $basecount(\ (sb_1, b_2)\ )$ | $= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | |

CC BY-NC-SA 2.0 Version November 6, 2026 (37)

# Predicate notation

**Notation**: for a predicate $P$ with domain $X_1 \times \cdots \times X_n$ and a $n$-tuple $(x_1, \ldots, x_n)$ with each $x_i \in X$, we can write $P(x_1, \ldots, x_n)$ to mean $P(\ (x_1, \ldots, x_n)\ )$.

# Predicates projecting example rna basecount

**New predicates from old**: Consider several different predicates that are related to $BC$:

1. Define the **new** predicate with domain $S \times B$ and rule

   $$basecount(\ (s, b)\ ) = 3$$

   Example domain element where predicate is $T$:

2. Define the **new** predicate with domain $S \times \mathbb{N}$ and rule

   $$basecount(\ (s, \mathtt{A})\ ) = n$$

   Example domain element where predicate is $T$:

3. Define the **new** predicate with domain $S \times B$ and rule

   $$\exists n \in \mathbb{N}\ (basecount(\ (s, b)\ ) = n)$$

   Example domain element where predicate is $T$:

4. Define the **new** predicate with domain $S$ and rule

   $$\forall b \in B\ (basecount(\ (s, b)\ ) = 1)$$

   Example domain element where predicate is $T$:

CC BY-NC-SA 2.0 Version November 6, 2026 (38)

# Nested quantifiers

### Nested quantifiers

$$\forall s \in S\ \forall b \in B\ \forall n \in \mathbb{N}\ (basecount(\ (s, b)\ ) = n)$$

In English:

Counterexample that proves this universal quantification is false:

$$\forall n \in \mathbb{N}\ \forall s \in S\ \forall b \in B\ (basecount(\ (s, b)\ ) = n)$$

In English:

Counterexample that proves this universal quantification is false:

CC BY-NC-SA 2.0 Version November 6, 2026 (39)

# Rna rnalen basecount definitions

*Recall the definitions*: The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

The function *rnalen* that computes the length of RNA strands in $S$ is defined recursively by:

| | | | |
|---|---|---|---|
| | | $rnalen : S$ | $\to \mathbb{Z}^+$ |
| Basis Step: | If $b \in B$ then | $rnalen(b)$ | $= 1$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then | $rnalen(sb)$ | $= 1 + rnalen(s)$ |

The function *basecount* that computes the number of a given base $b$ appearing in a RNA strand $s$ is defined recursively by:

| | | | | |
|---|---|---|---|---|
| | | $basecount : S \times B$ | $\to \mathbb{N}$ | |
| Basis Step: | If $b_1 \in B, b_2 \in B$ | $basecount(\ (b_1, b_2)\ )$ | $= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ | |
| Recursive Step: | If $s \in S, b_1 \in B, b_2 \in B$ | $basecount(\ (sb_1, b_2)\ )$ | $= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | |

# Structural induction motivating example rna

**Claim** $\forall s \in S$ ( $rnalen(s) > 0$ )

**Proof**: Let $s$ be an arbitrary RNA strand. By the recursive definition of $S$, either $s \in B$ or there is some strand $s_0$ and some base $b$ such that $s = s_0 b$. We will show that the inequality holds for both cases.

**Case**: Assume $s \in B$. We need to show $rnalen(s) > 0$. By the basis step in the definition of $rnalen$,

$$rnalen(s) = 1$$

which is greater than 0, as required.

**Case**: Assume there is some strand $s_0$ and some base $b$ such that $s = s_0 b$. We will show *(the stronger claim)* that

$$\forall u \in S \; \forall b \in B \; ( \; rnalen(u) > 0 \rightarrow rnalen(ub) > 0 \; )$$

Consider an arbitrary RNA strand $u$ and an arbitrary base $b$, and assume towards a direct proof, that

$$rnalen(u) > 0$$

We need to show that $rnalen(ub) > 0$.

$$rnalen(ub) = 1 + rnalen(u) > 1 + 0 = 1 > 0$$

as required.

# Proof strategies structural induction

**Proof by Structural Induction** To prove a universal quantification over a recursively defined set:

**Basis Step**: Show the statement holds for elements specified in the basis step of the definition.

**Recursive Step**: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

# Structural induction example rnalen basecount

**Claim** $\forall s \in S\,(rnalen(s) \geq basecount(\ (s, \mathtt{A})\ ))$:

**Proof**: We proceed by structural induction on the recursively defined set $S$.

**Basis Case**: We need to prove that the inequality holds for each element in the basis step of the recursive definition of $S$. Need to show

$$(\ rnalen(\mathtt{A}) \geq basecount(\ (\mathtt{A}, \mathtt{A})\ )\ ) \wedge (\ rnalen(\mathtt{C}) \geq basecount(\ (\mathtt{C}, \mathtt{A})\ )\ )$$
$$\wedge(\ rnalen(\mathtt{U}) \geq basecount(\ (\mathtt{U}, \mathtt{A})\ )\ ) \wedge (\ rnalen(\mathtt{G}) \geq basecount(\ (\mathtt{G}, \mathtt{A})\ )\ )$$

We calculate, using the definitions of *rnalen* and *basecount*:

**Recursive Case**: We will prove that

$$\forall u \in S\ \forall b \in B\ (\ rnalen(u) \geq basecount(\ (u, \mathtt{A})\ ) \rightarrow rnalen(ub) \geq basecount(\ (ub, \mathtt{A})\ )$$

Consider arbitrary RNA strand $u$ and arbitrary base $b$. Assume, as the **induction hypothesis**, that $rnalen(u) \geq basecount(\ (u, \mathtt{A})\ )$. We need to show that $rnalen(ub) \geq basecount(\ (ub, \mathtt{A})\ )$.

Using the recursive step in the definition of the function *rnalen*:

$$rnalen(ub) = 1 + rnalen(u)$$

The recursive step in the definition of the function *basecount* has two cases. We notice that $b = \mathtt{A} \vee b \neq \mathtt{A}$ and we proceed by cases.

*Case i.* Assume $b = \mathtt{A}$.

Using the first case in the recursive step in the definition of the function *basecount*:

$$basecount(\ (ub, \mathtt{A})\ ) = 1 + basecount(\ (u, \mathtt{A})\ )$$

By the **induction hypothesis**, we know that $basecount(\ (u, \mathtt{A})\ ) \leq rnalen(u)$ so:

$$basecount(\ (ub, \mathtt{A})\ ) = 1 + basecount(\ (u, \mathtt{A})\ ) \leq 1 + rnalen(u) = rnalen(ub)$$

and, thus, $basecount(\ (ub, \mathtt{A})\ ) \leq rnalen(ub)$, as required.

*Case ii.* Assume $b \neq \mathtt{A}$.

Using the second case in the recursive step in the definition of the function *basecount*:

$$basecount(\ (ub, \mathtt{A})\ ) = basecount(\ (u, \mathtt{A})\ )$$

By the **induction hypothesis**, we know that $basecount(\ (u, \texttt{A})\ ) \leq rnalen(u)$ so:

$$basecount(\ (ub, \texttt{A})\ ) = basecount(\ (u, \texttt{A})\ ) \leq rnalen(u) < 1 + rnalen(u) = rnalen(ub)$$

and, thus, $basecount(\ (ub, \texttt{A})\ ) \leq rnalen(ub)$, as required.

# Proofs signposting kinds of claims

To organize our proofs, it's useful to highlight which claims are most important for our overall goals. We use some terminology to describe different roles statements can have.

**Theorem**: Statement that can be shown to be true, usually an important one.

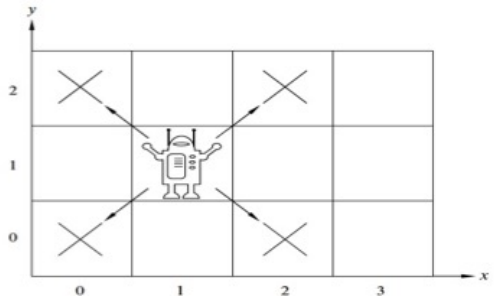
Less important theorems can be called **proposition**, **fact**, **result**, **claim**.

**Lemma**: A less important theorem that is useful in proving a theorem.

**Corollary**: A theorem that can be proved directly after another one has been proved, without needing a lot of extra work.

**Invariant**: A theorem that describes a property that is true about an algorithm or system no matter what inputs are used.

# Structural induction example robot grid




**Theorem**: A robot on an infinite 2-dimensional integer grid starts at $(0, 0)$ and at each step moves to diagonally adjacent grid point. This robot can / cannot (*circle one*) reach $(1, 0)$.

**Definition** The set of positions the robot can visit $Pos$ is defined by:

Basis Step: $(0, 0) \in Pos$
Recursive Step: If $(x, y) \in Pos$, then

are also in $Pos$

*Example elements of Pos are*:

**Lemma**: $\forall(x, y) \in Pos$ ($x + y$ is an even integer )

*Why are we calling this a lemma?*

Proof of theorem using lemma: To show is $(1, 0) \notin Pos$. Rewriting the lemma to explicitly restrict the domain of the universal, we have $\forall(x, y)$ ( $(x, y) \in Pos \rightarrow (x+y$ is an even integer) ). Since the universal is true, ( $(1, 0) \in Pos \rightarrow (1 + 0$ is an even integer) ) is a true statement. Evaluating the conclusion of this conditional statement: By definition of long division, since $1 = 0 \cdot 2 + 1$ (where $0 \in \mathbb{Z}$ and $1 \in \mathbb{Z}$ and $0 \leq 1 < 2$ mean that 0 is the quotient and 1 is the remainder), $1$ **mod** $2 = 1$ which is not 0 so the conclusion is false. A true conditional with a false conclusion must have a false hypothesis: $(1, 0) \notin Pos$, QED. □

Proof of lemma by structural induction:

**Basis Step**:

**Recursive Step**: Consider arbitrary $(x, y) \in Pos$. To show is:

($x + y$ is an even integer) → (sum of coordinates of next position is even integer)

Assume **as the induction hypothesis, IH** that:

# Proof strategy mathematical induction

**Proof by Mathematical Induction**
To prove a universal quantification over the set of all integers greater than or equal to some base integer $b$,

**Basis Step**: Show the property holds for $b$.

**Recursive Step**: Consider an arbitrary integer $n$ greater than or equal to $b$, assume (as the **induction hypothesis**) that the property holds for $n$, and use this and other facts to prove that the property holds for $n + 1$.

CC BY-NC-SA 2.0 Version November 6, 2026 (45)

# Structural induction example sum of powers

The set $\mathbb{N}$ is recursively defined. Therefore, the function $sumPow : \mathbb{N} \to \mathbb{N}$ which computes, for input $i$, the sum of the nonnegative powers of 2 up to and including exponent $i$ is defined recursively by

Basis step: $sumPow(0) = 1$

Recursive step: If $x \in \mathbb{N}$, then $sumPow(x+1) = sumPow(x) + 2^{x+1}$

$sumPow(0) =$

$sumPow(1) =$

$sumPow(2) =$

Fill in the blanks in the following proof of

$$\forall n \in \mathbb{N}\ (sumPow(n) = 2^{n+1} - 1)$$

**Proof**: Since $\mathbb{N}$ is recursively defined, we proceed by _____.

**Basis case**: We need to show that _____. Evaluating each side: $LHS = sumPow(0) = 1$ by the basis case in the recursive definition of $sumPow$; $RHS = 2^{0+1} - 1 = 2^1 - 1 = 2 - 1 = 1$. Since $1 = 1$, the equality holds.

**Recursive case**: Consider arbitrary natural number $n$ and assume, as the _____ that $sumPow(n) = 2^{n+1} - 1$. We need to show that _____. Evaluating each side:

$$LHS = sumPow(n+1) \overset{\text{rec def}}{=} sumPow(n) + 2^{n+1} \overset{\text{IH}}{=} (2^{n+1} - 1) + 2^{n+1}.$$

$$RHS = 2^{(n+1)+1} - 1 \overset{\text{exponent rules}}{=} 2 \cdot 2^{n+1} - 1 = \left(2^{n+1} + 2^{n+1}\right) - 1 \overset{\text{regrouping}}{=} (2^{n+1} - 1) + 2^{n+1}$$

Thus, $LHS = RHS$. The structural induction is complete and we have proved the universal generalization. □

CC BY-NC-SA 2.0 Version November 6, 2026 (46)

# Proof strategy strong induction

**Proof by Strong Induction**
To prove that a universal quantification over the set of all integers greater than or equal to some base integer $b$ holds, pick a fixed nonnegative integer $j$ and then:

| | |
|---|---|
| **Basis Step**: | Show the statement holds for $b$, $b+1$, $\ldots$, $b+j$. |
| **Recursive Step**: | Consider an arbitrary integer $n$ greater than or equal to $b+j$, assume (as the **strong induction hypothesis**) that the property holds for **each of** $b$, $b+1$, $\ldots$, $n$, and use this and other facts to prove that the property holds for $n+1$. |

# Binary expansions exist proof

**Theorem**: Every positive integer is a sum of (one or more) distinct powers of 2. *Binary expansions exist!*

The idea in the "Least significant first" algorithm for computing binary expansions is that the binary expansion of *half* a number becomes *part* of the binary expansion of the number of itself. We can use this idea in a proof by strong induction that binary expansions exist for all positive integers $n$.

**Proof by strong induction**, with $b = 1$ and $j = 0$. We need to prove that for each positive integer $n$, there is a positive integer $k$ and coefficients $a_0, \ldots, a_{k-1}$ where each $a_i$ is 0 or 1 and $a_{k-1} \neq 0$, and

$$n = \sum_{i=0}^{k-1} a_i 2^i$$

**Basis step**: WTS property is true about 1.

**Recursive step**: Consider an arbitrary integer $n \geq 1$.

Assume (as the strong induction hypothesis, IH) that the property is true about each of $1, \ldots, n$.

WTS that the property is true about $n+1$.

*Idea*: We will apply the IH to $(n+1)$ **div** 2.

*Why is this ok?*

*Why is this helpful?*

By the IH, we can write $(n+1)$ **div** 2 as a sum of powers of 2. In other words, there are values $a_{k-1}, \ldots, a_0$ such that each $a_i$ is 0 or 1, $a_{k-1} = 1$, and

$$\sum_{i=0}^{k-1} a_i 2^i = (n+1) \textbf{ div } 2$$

Define the collection of coefficients

$$c_j = \begin{cases} a_{j-1} & \text{if } 1 \leq j \leq k \\ (n+1) \textbf{ mod } 2 & \text{if } j = 0 \end{cases}$$

Calculating:

$$\begin{aligned}
\sum_{j=0}^{k} c_j 2^j &= c_0 + \sum_{j=1}^{k} c_j 2^j = c_0 + \sum_{i=0}^{k-1} c_{i+1} 2^{i+1} && \text{re-indexing the summation} \\
&= c_0 + 2 \cdot \sum_{i=0}^{k-1} c_{i+1} 2^i && \text{factoring out a 2 from each term in the sum} \\
&= c_0 + 2 \cdot \sum_{i=0}^{k-1} a_i 2^i && \text{by definition of } c_{i+1} \\
&= c_0 + 2\,(\ (n+1) \textbf{ div } 2\ ) && \text{by IH} \\
&= (\ (n+1) \textbf{ mod } 2\ ) + 2\,(\ (n+1) \textbf{ div } 2\ ) && \text{by definition of } c_0 \\
&= n+1 && \text{by definition of long division}
\end{aligned}$$

Thus, $n+1$ can be expressed as a sum of powers of 2, as required.

# Fundamental theorem proof

**Theorem**: Every positive integer *greater than 1* is a product of (one or more) primes.

**Before we prove, let's try some examples**:

$20 =$

$100 =$

$5 =$

**Proof by strong induction**, with $b = 2$ and $j = 0$.

**Basis step**: WTS property is true about 2.

Since 2 is itself prime, it is already written as a product of (one) prime.

**Recursive step**: Consider an arbitrary integer $n \geq 2$. Assume (as the strong induction hypothesis, IH) that the property is true about each of $2, \ldots, n$. WTS that the property is true about $n + 1$: We want to show that $n + 1$ can be written as a product of primes. Notice that $n + 1$ is itself prime or it is composite.

*Case 1*: assume $n + 1$ is prime and then immediately it is written as a product of (one) prime so we are done.

*Case 2*: assume that $n + 1$ is composite so there are integers $x$ and $y$ where $n + 1 = xy$ and each of them is between 2 and $n$ (inclusive). Therefore, the induction hypothesis applies to each of $x$ and $y$ so each of these factors of $n + 1$ can be written as a product of primes. Multiplying these products together, we get a product of primes that gives $n + 1$, as required.

Since both cases give the necessary conclusion, the proof by cases for the recursive step is complete.

CC BY-NC-SA 2.0 Version November 6, 2026 (49)

# Strong induction making change proof idea

Suppose we had postage stamps worth 5 cents and 3 cents. Which number of cents can we form using these stamps? In other words, which postage can we pay?

11?

15?

4?

$$CanPay(0) \wedge \neg CanPay(1) \wedge \neg CanPay(2) \wedge$$
$$CanPay(3) \wedge \neg CanPay(4) \wedge CanPay(5) \wedge CanPay(6)$$
$$\neg CanPay(7) \wedge \forall n \in \mathbb{Z}^{\geq 8} CanPay(n)$$

where the predicate $CanPay$ with domain $\mathbb{N}$ is

$$CanPay(n) = \exists x \in \mathbb{N} \exists y \in \mathbb{N}(5x + 3y = n)$$

**Proof** (idea): First, explicitly give witnesses or general arguments for postages between 0 and 7. To prove the universal claim, we can use mathematical induction or strong induction.

*Approach 1, mathematical induction*: if we have stamps that add up to $n$ cents, need to use them (and others) to give $n + 1$ cents. How do we get 1 cent with just 3-cent and 5-cent stamps?
Either take away a 5-cent stamps and add two 3-cent stamps,
or take away three 3-cent stamps and add two 5-cent stamps.
The details of this proof by mathematical induction are making sure we have enough stamps to use one of these approaches.

*Approach 2, strong induction*: assuming we know how to make postage for **all** smaller values (greater than or equal to 8), when we need to make $n+1$ cents, add one 3 cent stamp to however we make $(n + 1) - 3$ cents.
The details of this proof by strong induction are making sure we stay in the domain of the universal when applying the induction hypothesis.

CC BY-NC-SA 2.0 Version November 6, 2026 (50)

# Strong induction nim

Consider the following game: two players start with two (equal) piles of jellybeans in front of them. They take turns removing any positive integer number of jellybeans at a time from one of two piles in front of them in turns.

The player who removes the last jellybean wins the game.

Which player (if any) has a strategy to guarantee to win the game?

Try out some games, starting with 1 jellybean in each pile, then 2 jellybeans in each pile, then 3 jellybeans in each pile. Who wins in each game?

Notice that reasoning about the strategy for the 1 jellybean game is easier than about the strategy for the 2 jellybean game.

*Formulate a winning strategy by working to transform the game to a simpler one we know we can win.*

CC BY-NC-SA 2.0 Version November 6, 2026 (51)

*Player 2's Strategy*: Take the same number of jellybeans that Player 1 did, but from the opposite pile.

*Why is this a good idea*: If Player 2 plays this strategy, at the next turn Player 1 faces a game with the same setup as the original, just with fewer jellybeans in the two piles. Then Player 2 can keep playing this strategy to win.

**Claim**: Player 2's strategy guarantees they will win the game.

**Proof**: By strong induction, we will prove that for all positive integers $n$, Player 2's strategy guarantees a win in the game that starts with $n$ jellybeans in each pile.

**Basis step**: WTS Player 2's strategy guarantees a win when each pile starts with 1 jellybean.

In this case, Player 1 has to take the jellybean from one of the piles (because they can't take from both piles at once). Following the strategy, Player 2 takes the jellybean from the other pile, and wins because this is the last jellybean.

**Recursive step**: Let $n$ be a positive integer. As the strong induction hypothesis, assume that Player 2's strategy guarantees a win in the games where there are $1, 2, \ldots, n$ many jellybeans in each pile at the start of the game.

WTS that Player 2's strategy guarantees a win in the game where there are $n+1$ jellybeans in each pile at the start of the game.

In this game, the first move has Player 1 take some number, call it $c$ (where $1 \leq c \leq n+1$), of jellybeans from one of the piles. Playing according to their strategy, Player 2 then takes the same number of jellybeans from the other pile.

Notice that $(c = n+1) \vee (c \leq n)$.

*Case 1*: Assume $c = n+1$, then in their first move, Player 2 wins because they take all of the second pile, which includes the last jellybean.

*Case 2*: Assume $c \leq n$. Then after Player 2's first move, the two piles have an equal number of jellybeans. The number of jellybeans in each pile is

$$(n+1) - c$$

and, since $1 \leq c \leq n$, this number is between 1 and $n$. Thus, at this stage of the game, the game appears identical to a new game where the two piles have an equal number of jellybeans between 1 and $n$. Thus, the strong induction hypothesis applies, and Player 2's strategy guarantees they win.

# Linked lists definition

**Definition** The set of linked lists of natural numbers $L$ is defined recursively by

| | |
|---|---|
| Basis Step: | $[] \in L$ |
| Recursive Step: | If $l \in L$ and $n \in \mathbb{N}$, then $(n, l) \in L$ |

# Linked lists examples

Visually:

Example: the list with two nodes whose first node has 20 and whose second node has 42

# Linked list length definition

**Definition**: The length of a linked list of natural numbers $L$, $length : L \to \mathbb{N}$ is defined by

| | | | |
|---|---|---|---|
| Basis Step: | | $length(\ []\ )$ | $= 0$ |
| Recursive Step: | If $l \in L$ and $n \in \mathbb{N}$, then | $length(\ (n, l)\ )$ | $= 1 + length(l)$ |

# Linked lists prepend definition

**Definition**: The function $prepend : L \times \mathbb{N} \to L$ that adds an element at the front of a linked list is defined by

# Linked list append definition

**Definition** The function $append : L \times \mathbb{N} \to L$ that adds an element at the end of a linked list is defined by

| | |
|---|---|
| Basis Step: | If $m \in \mathbb{N}$ then |
| Recursive Step: | If $l \in L$ and $n \in \mathbb{N}$ and $m \in \mathbb{N}$, then |

# Linked list append length claim proof

**Claim**: $\forall l \in L$ ( $length($ $append($ $(l, 100)$ ) ) $> length(l)$ )

**Proof:** By structural induction on $L$, we have two cases:

**Basis Step**

| | |
|---|---|
| 1. **To Show** $length($ $append($ $([], 100)$ ) ) $> length($ [] ) | Because [] is the only element defined in the basis step of $L$, we only need to prove that the property holds for []. |
| 2. **To Show** $length($ $(100, [])$ ) $> length($ [] ) | By basis step in definition of *append*. |
| 3. **To Show** $(1 + length($ [] )) $> length($ [] ) | By recursive step in definition of *length*. |
| 4. **To Show** $1 + 0 > 0$ | By basis step in definition of *length*. |
| 5. $T$ | By properties of integers |
| QED | Because we got to $T$ only by rewriting **To Show** to equivalent statements, using well-defined proof techniques, and applying definitions. |

**Recursive Step**

Consider an arbitrary: $l' \in L$, $n \in \mathbb{N}$, and we assume as the **induction hypothesis** that:

$$length($ $append($ $(l', 100$ ) ) $> length(l')$$

Our goal is to show that $length($ $append($ ( $(n, l'), 100$ ) ) ) $> length($ $(n, l')$ ) is also true. We start by working with one side of the candidate inequality:

$$\begin{aligned}
LHS &= length($ $append($ ( $(n, l'), 100$ ) ) ) \\
&= length($ $(n, append($ $(l', 100)$ ) ) ) \qquad \text{by the recursive definition of } append \\
&= 1 + length($ $append($ $(l', 100)$ ) ) \qquad \text{by the recursive definition of } length \\
&> 1 + length(l') \qquad \text{by the induction hypothesis} \\
&= length((n, l')) \qquad \text{by the recursive definition of } length \\
&= RHS
\end{aligned}$$

CC BY-NC-SA 2.0 Version November 6, 2026 (54)

## Linked list example each length

Prove or disprove: $\forall n \in \mathbb{N}\ \exists l \in L\ (\ length(l) = n\ )$

## Proof strategy proof by contradiction

**New! Proof by Contradiction**
To prove that a statement $p$ is true, pick another statement $r$ and once we show that $\neg p \to (r \wedge \neg r)$ then we can conclude that $p$ is true.
*Informally* The statement we care about can't possibly be false, so it must be true.

# Least greatest proofs

For a set of numbers $X$, how do you formalize "there is a greatest $X$" or "there is a least $X$"?

**Prove** or **disprove**: There is a least prime number.

**Prove** or **disprove**: There is a greatest integer.

*Approach 1, De Morgan's and universal generalization*:

*Approach 2, proof by contradiction*:

*Extra examples*: Prove or disprove that $\mathbb{N}$, $\mathbb{Q}$ each have a least and a greatest element.

CC BY-NC-SA 2.0 Version November 6, 2026 (56)

# Gcd definition

**Definition**: **Greatest common divisor** Let $a$ and $b$ be integers, not both zero. The largest integer $d$ such that $d$ is a factor of $a$ and $d$ is a factor of $b$ is called the greatest common divisor of $a$ and $b$ and is denoted by $gcd(\ (a, b)\ )$.

# Gcd examples

Why do we restrict to the situation where $a$ and $b$ are not both zero?

Calculate $gcd(\ (10, 15)\ )$

Calculate $gcd(\ (10, 20)\ )$

CC BY-NC-SA 2.0 Version November 6, 2026 (57)

# Gcd basic claims

**Claim**: For any integers $a, b$ (not both zero), $gcd(\ (a, b)\ ) \geq 1$.

**Proof**: *Show that* 1 *is a common factor of any two integers, so since the gcd is the greatest common factor it is greater than or equal to any common factor.*

**Claim**: For any positive integers $a, b$, $gcd(\ (a, b)\ ) \leq a$ and $gcd(\ (a, b)\ ) \leq b$.

**Proof** *Using the definition of gcd and the fact that factors of a positive integer are less than or equal to that integer.*

**Claim**: For any positive integers $a, b$, if $a$ divides $b$ then $gcd(\ (a, b)\ ) = a$.

**Proof** *Using previous claim and definition of gcd.*

**Claim**: For any positive integers $a, b, c$, if there is some integer $q$ such that $a = bq + c$,

$$gcd(\ (a, b)\ ) = gcd(\ (b, c)\ )$$

CC BY-NC-SA 2.0 Version November 6, 2026 (58)

**Proof** *Prove that any common divisor of $a, b$ divides $c$ and that any common divisor of $b, c$ divides $a$.*

# Gcd lemma relatively prime

**Lemma**: For any integers $p, q$ (not both zero), $gcd\left( \left( \frac{p}{gcd(\ (p,q)\ )}, \frac{q}{gcd(\ (p,q)\ )} \right) \right) = 1$ . In other words, can reduce to relatively prime integers by dividing by gcd.

**Proof**:

Let $x$ be arbitrary positive integer and assume that $x$ is a factor of each of $\frac{p}{gcd(\ (p,q)\ )}$ and $\frac{q}{gcd(\ (p,q)\ )}$. This gives integers $\alpha$, $\beta$ such that

$$\alpha x = \frac{p}{gcd(\ (p,q)\ )} \qquad \beta x = \frac{q}{gcd(\ (p,q)\ )}$$

Multiplying both sides by the denominator in the RHS:

$$\alpha x \cdot gcd(\ (p,q)\ ) = p \qquad \beta x \cdot gcd(\ (p,q)\ ) = q$$

In other words, $x \cdot gcd(\ (p,q)\ )$ is a common divisor of $p, q$. By definition of $gcd$, this means

$$x \cdot gcd(\ (p,q)\ ) \leq gcd(\ (p,q)\ )$$

and since $gcd(\ (p,q)\ )$ is positive, this means, $x \leq 1$.

CC BY-NC-SA 2.0 Version November 6, 2026 (60)

# Rational numbers definition

The **set of rational numbers**, $\mathbb{Q}$ is defined as

$$\left\{ \frac{p}{q} \mid p \in \mathbb{Z} \text{ and } q \in \mathbb{Z} \text{ and } q \neq 0 \right\} \quad \text{or, equivalently,} \quad \{x \in \mathbb{R} \mid \exists p \in \mathbb{Z} \exists q \in \mathbb{Z}^{+} (p = x \cdot q)\}$$

*Extra practice*: Use the definition of set equality to prove that the definitions above give the same set.

# Proof by contradiction irrational

**Goal**: The square root of 2 is not a rational number. In other words: $\neg\exists x \in \mathbb{Q}(x^2 - 2 = 0)$

**Attempted proof**: The definition of the set of rational numbers is the collection of fractions $p/q$ where $p$ is an integer and $q$ is a nonzero integer. Looking for a **witness** $p$ and $q$, we can write the square root of 2 as the fraction $\sqrt{2}/1$, where 1 is a nonzero integer. Since the numerator is not in the domain, this witness is not allowed, and we have shown that the square root of 2 is not a fraction of integers (with nonzero denominator). Thus, the square root of 2 is not rational.

*The problem in the above attempted proof is that* _____

**Lemma 1:** For every two integers $a$ and $b$, not both zero, with $gcd(\ (a,b)\ ) = 1$, it is not the case that both $a$ is even and $b$ is even.

**Lemma 2:** For every integer $x$, $x$ is even if and only if $x^2$ is even.

**Proof**: Towards a proof by contradiction, we will define a statement $r$ such that $\sqrt{2} \in \mathbb{Q} \to (r \wedge \neg r)$.

Assume that $\sqrt{2} \in \mathbb{Q}$. Namely, there are positive integers $p, q$ such that

$$\sqrt{2} = \frac{p}{q}$$

Let $a = \frac{p}{gcd(\ (p,q)\ )}$, $b = \frac{q}{gcd(\ (p,q)\ )}$, then

$$\sqrt{2} = \frac{a}{b} \qquad \text{and} \qquad gcd(\ (a,b)\ ) = 1$$

By Lemma 1, $a$ and $b$ are not both even. We define $r$ to be the statement "$a$ is even and $b$ is even", and we have proved $\neg r$.

Squaring both sides and clearing denominator: $2b^2 = a^2$.

By definition of even, since $b^2$ is an integer, $a^2$ is even.

By Lemma 2, this guarantees that $a$ is even too. So, by definition of even, there is some integer (call it $c$), such that $a = 2c$.

Plugging into the equation:

$$2b^2 = a^2 = (2c)^2 = 4c^2$$

and dividing both sides by 2

$$b^2 = 2c^2$$

and since $c^2$ is an integer, $b^2$ is even. By Lemma 2, $b$ is even too. Thus, $a$ is even and $b$ is even and we have proved $r$.

In other words, assuming that $\sqrt{2} \in \mathbb{Q}$ guarantees $r \wedge \neg r$, which is impossible, so $\sqrt{2} \notin \mathbb{Q}$. QED

CC BY-NC-SA 2.0 Version November 6, 2026 (62)

# Finite sets definition

**Definition**: A **finite** set is one whose distinct elements can be counted by a natural number.

# Cardinality motivation

**Motivating question**: when can we say one set is *bigger than* another?

*Does the definition change when we're comparing finite sets? infinite sets?*

Which is bigger?

- The set $\{1, 2, 3\}$ or the set $\{0, 1, 2, 3\}$?
- The set $\{0, \pi, \sqrt{2}\}$ or the set $\{\mathbb{N}, \mathbb{R}, \emptyset\}$?
- The set $\mathbb{N}$ or the set $\mathbb{Q}$?
- The set $\mathbb{Q}$ or the set $\mathbb{R}$?
- The set $\mathbb{N}$ or the set $\mathbb{R}^+$?

# Cardinality rationale for functions

**Key idea for cardinality**: Counting distinct elements is a way of labelling elements with natural numbers. This is a function! In general, functions let us associate elements of one set with those of another. If the association is "*good*", we get a correspondence between the elements of the subsets which can relate the sizes of the sets.

# Musical chairs analogy

*Analogy*: Musical chairs



People try to sit down when the music stops

Person☼ sits in Chair 1, Person☺ sits in Chair 2,

Person☹ is left standing!

CC BY-NC-SA 2.0 Version November 6, 2026 (63)

What does this say about the number of chairs and the number of people?

## Well defined functions

Recall that a function is defined by its (1) domain, (2) codomain, and (3) rule assigning each element in the domain exactly one element in the codomain. The domain and codomain are nonempty sets. The rule can be depicted as a table, formula, English description, etc.

A function can *fail to be well-defined* if there is some domain element where the function rule doesn't give a unique codomain element. For example, the function rule might lead to more than one potential image, or to an image outside of the codomain.

*Example*: $f_A : \mathbb{R}^+ \to \mathbb{Q}$ with $f_A(x) = x$ is **not** a well-defined function because

*Example*: $f_B : \mathbb{Q} \to \mathbb{Z}$ with $f_B\left(\frac{p}{q}\right) = p + q$ is **not** a well-defined function because

*Example*: $f_C : \mathbb{Z} \to \mathbb{R}$ with $f_C(x) = \frac{x}{|x|}$ is **not** a well-defined function because

## Injective function definition

**Definition** : A function $f : D \to C$ is **one-to-one** (or injective) means for every $a, b$ in the domain $D$, if $f(a) = f(b)$ then $a = b$.

Formally, $f : D \to C$ is one-to-one means _____.

## Injective functions visually

Informally, a function being one-to-one means "no duplicate images".

## Cardinality lower bound definition

**Definition**: For nonempty sets $A, B$, we say that **the cardinality of $A$ is no bigger than the cardinality of** $B$, and write $|A| \leq |B|$, to mean there is a one-to-one function with domain $A$ and codomain $B$. Also, we define $|\emptyset| \leq |B|$ for all sets $B$.

## Injective cardinality musical chairs

*In the analogy*: The function $sitter : \{Chair1, Chair2\} \to \{Person☼, Person☺, Person☹\}$ given by $sitter(Chair1) = Person☼$, $sitter(Chair2) = Person☺$, is one-to-one and witnesses that

$$|\{Chair1, Chair2\}| \leq |\{Person☼, Person☺, Person☹\}|$$

## Surjective function definition

**Definition**: A function $f : D \to C$ is **onto** (or surjective) means for every $b$ in the codomain, there is an element $a$ in the domain with $f(a) = b$.

Formally, $f : D \to C$ is onto means _____.

CC BY-NC-SA 2.0 Version November 6, 2026 (65)

## Surjective functions visually

Informally, a function being onto means "every potential image is an actual image".

## Cardinality upper bound definition

**Definition**: For nonempty sets $A, B$, we say that **the cardinality of $A$ is no smaller than the cardinality of** $B$, and write $|A| \geq |B|$, to mean there is an onto function with domain $A$ and codomain $B$. Also, we define $|A| \geq |\emptyset|$ for all sets $A$.

## Surjective cardinality musical chairs

*In the analogy*: The function $triedToSit : \{Person☼, Person☺, Person☹\} \to \{Chair1, Chair2\}$ given by $triedToSit(Person☼) = Chair1$, $triedToSit(Person☺) = Chair2$, $triedToSit(Person☹) = Chair2$, is onto and witnesses that

$$|\{Person☼, Person☺, Person☹\}| \geq |\{Chair1, Chair2\}|$$

## Bijection definition

**Definition** : A function $f : D \to C$ is a **bijection** means that it is both one-to-one and onto. The **inverse** of a bijection $f : D \to C$ is the function $g : C \to D$ such that $g(b) = a$ iff $f(a) = b$.

# Compound proposition definitions

**Proposition**: Declarative sentence that is true or false (not both).

**Propositional variable**: Variable that represents a proposition.

**Compound proposition**: New proposition formed from existing propositions (potentially) using logical operators. *Note*: A propositional variable is one example of a compound proposition.

**Truth table**: Table with one row for each of the possible combinations of truth values of the input and an additional column that shows the truth value of the result of the operation corresponding to a particular row.

# Logical equivalence

**Logical equivalence** : Two compound propositions are **logically equivalent** means that they have the same truth values for all settings of truth values to their propositional variables.

**Tautology**: A compound proposition that evaluates to true for all settings of truth values to its propositional variables; it is abbreviated $T$.

**Contradiction**: A compound proposition that evaluates to false for all settings of truth values to its propositional variables; it is abbreviated $F$.

**Contingency**: A compound proposition that is neither a tautology nor a contradiction.

# Tautology contradiction contingency examples

Label each of the following as a tautology, contradiction, or contingency.

$p \wedge p$

$p \oplus p$

$p \vee p$

$p \vee \neg p$

$p \wedge \neg p$

# Logical equivalence extra example

*Extra Example*: Which of the compound propositions in the table below are logically equivalent?

| Input | | Output | | | | |
|---|---|---|---|---|---|---|
| $p$ | $q$ | $\neg(p \wedge \neg q)$ | $\neg(\neg p \vee \neg q)$ | $(\neg p \vee q)$ | $(\neg q \vee \neg p)$ | $(p \wedge q)$ |
| $T$ | $T$ | | | | | |
| $T$ | $F$ | | | | | |
| $F$ | $T$ | | | | | |
| $F$ | $F$ | | | | | |

# Logical equivalence identities andornotxor

**(Some) logical equivalences**

*Can replace $p$ and $q$ with any compound proposition*

$\neg(\neg p) \equiv p$ **Double negation**

$p \vee q \equiv q \vee p$ $p \wedge q \equiv q \wedge p$ **Commutativity** Ordering of terms

$(p \vee q) \vee r \equiv p \vee (q \vee r)$ $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$ **Associativity** Grouping of terms

$p \wedge F \equiv F$ $p \vee T \equiv T$ $p \wedge T \equiv p$ $p \vee F \equiv p$ **Domination** aka short circuit evaluation

$\neg(p \wedge q) \equiv \neg p \vee \neg q$ $\neg(p \vee q) \equiv \neg p \wedge \neg q$ **DeMorgan's Laws**

# Fixed width example

| Decimal $b = 10$ | Binary $b = 2$ | Binary fixed-width 10 $b = 2$, $w = 10$ | Binary fixed-width 7 $b = 2$, $w = 7$ | Binary fixed-width 4 $b = 2$, $w = 4$ |
|---|---|---|---|---|
| $(20)_{10}$ | | | | |

# Fixed width fractional

**Definition** For $b$ an integer greater than 1, $w$ a positive integer, $w'$ a positive integer, and $x$ a real number the **base $b$ fixed-width expansion of $x$ with integer part width $w$ and fractional part width $w'$** is $(a_{w-1} \cdots a_1 a_0 . c_1 \cdots c_{w'})_{b,w,w'}$ where $a_0, a_1, \ldots, a_{w-1}, c_1, \ldots, c_{w'}$ are nonnegative integers less than $b$ and

$$x \geq \sum_{i=0}^{w-1} a_i b^i + \sum_{j=1}^{w'} c_j b^{-j} \qquad \text{and} \qquad x < \sum_{i=0}^{w-1} a_i b^i + \sum_{j=1}^{w'} c_j b^{-j} + b^{-w'}$$

| | |
|---|---|
| 3.75 in fixed-width binary, integer part width 2, fractional part width 8 | |
| 0.1 in fixed-width binary, integer part width 2, fractional part width 8 | |

```
|welcome $jshell
|  Welcome to JShell -- Version 10.0.1
|  For an introduction type: /help intro

jshell> 0.1
$1 ==>

jshell> 0.2
$2 ==>

jshell> 0.1 + 0.2
$3 ==>

jshell> Math.sqrt(2)
$4 ==>

jshell> Math.sqrt(2)*Math.sqrt(2)
$5 ==>

jshell>
```

Note: Java uses floating point, not fixed width representation, but similar rounding errors appear in both.

# Negative int expansions

**Representing negative integers in binary**: Fix a positive integer width for the representation $w$, $w > 1$.

| | To represent a positive integer $n$ | To represent a negative integer $-n$ |
|---|---|---|
| Sign-magnitude | $[0a_{w-2} \cdots a_0]_{s,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$<br><br>Example $n = 17$, $w = 7$: | $[1a_{w-2} \cdots a_0]_{s,w}$ , where $n = (a_{w-2} \cdots a_0)_{2,w-1}$<br><br>Example $-n = -17$, $w = 7$: |
| 2s complement | $[0a_{w-2} \cdots a_0]_{2c,w}$, where $n = (a_{w-2} \cdots a_0)_{2,w-1}$<br><br>Example $n = 17$, $w = 7$: | $[1a_{w-2} \cdots a_0]_{2c,w}$, where $2^{w-1} - n = (a_{w-2} \cdots a_0)_{2,w-1}$<br><br>Example $-n = -17$, $w = 7$: |

# Calculating 2s complement

For positive integer $n$, to represent $-n$ in 2s complement with width $w$,

- Calculate $2^{w-1} - n$, convert result to binary fixed-width $w - 1$, pad with leading 1, or
- Express $-n$ as a sum of powers of 2, where the leftmost $2^{w-1}$ is negative weight, or
- Convert $n$ to binary fixed-width $w$, flip bits, add 1 (ignore overflow)

*Challenge: use definitions to explain why each of these approaches works.*

CC BY-NC-SA 2.0 Version November 6, 2026 (70)

# Netflix intro

What data should we encode about each Netflix account holder to help us make effective recommendations?

In machine learning, clustering can be used to group similar data for prediction and recommendation. For example, each Netflix user's viewing history can be represented as a $n$-tuple indicating their preferences about movies in the database, where $n$ is the number of movies in the database. People with similar tastes in movies can then be clustered to provide recommendations of movies for one another. Mathematically, clustering is based on a notion of distance between pairs of $n$-tuples.

# Data types

| **Term** | **Examples**:<br>(add additional examples from class) |
|---|---|
| **set**<br>unordered collection of elements<br>*repetition doesn't matter*<br>*Equal sets agree on membership of all elements* | $7 \in \{43, 7, 9\}$ $2 \notin \{43, 7, 9\}$ |
| **$n$-tuple**<br>ordered sequence of elements with $n$ "slots" $(n > 0)$<br>*repetition matters, fixed length*<br>*Equal $n$-tuples have corresponding components equal* | |
| **string**<br>ordered finite sequence of elements each from specified set (called the alphabet over which the string is defined)<br>*repetition matters, arbitrary finite length*<br>*Equal strings have same length and corresponding characters equal* | |

*Special cases*:

When $n = 2$, the 2-tuple is called an **ordered pair**.

A string of length 0 is called the **empty string** and is denoted $\lambda$.

A set with no elements is called the **empty set** and is denoted $\{\}$ or $\emptyset$.

# Set operations

To define a set we can use the roster method, set builder notation, a recursive definition, and also we can apply a set operation to other sets.

**New! Cartesian product of sets** and **set-wise concatenation of sets of strings**

**Definition**: Let $X$ and $Y$ be sets. The **Cartesian product** of $X$ and $Y$, denoted $X \times Y$, is the set of all ordered pairs $(x, y)$ where $x \in X$ and $y \in Y$

$$X \times Y = \{(x, y) \mid x \in X \text{ and } y \in Y\}$$

Conventions: (1) Cartesian products can be chained together to result in sets of $n$-tuples and (2) When we form the Cartesian product of a set with itself $X \times X$ we can denote that set as $X^2$, or $X^n$ for the Cartesian product of a set with itself $n$ times for a positive integer $n$.

**Definition**: Let $X$ and $Y$ be sets of strings over the same alphabet. The **set-wise concatenation** of $X$ and $Y$, denoted $X \circ Y$, is the set of all results of string concatenation $xy$ where $x \in X$ and $y \in Y$

$$X \circ Y = \{xy \mid x \in X \text{ and } y \in Y\}$$

**Pro-tip**: the meaning of writing one element next to another like $xy$ depends on the data-types of $x$ and $y$. When $x$ and $y$ are strings, the convention is that $xy$ is the result of string concatenation. When $x$ and $y$ are numbers, the convention is that $xy$ is the result of multiplication. This is (one of the many reasons) why is it very important to declare the data-type of variables before we use them.

*Fill in the missing entries in the table*:

| Set | Example elements in this set and their data type: |
|---|---|
| $B$ | `A` `C` `G` `U` |
| | $(\texttt{A}, \texttt{C})$ $(\texttt{U}, \texttt{U})$ |
| $B \times \{-1, 0, 1\}$ | |
| $\{-1, 0, 1\} \times B$ | |
| | $(0, 0, 0)$ |
| $\{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{U}\} \circ \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{U}\}$ | |
| | `GGGG` |

# Defining functions

**New! Defining functions** A function is defined by its (1) domain, (2) codomain, and (3) rule assigning each element in the domain exactly one element in the codomain.

The domain and codomain are nonempty sets.
The rule can be depicted as a table, formula, piecewise definition, or English description.
The notation is

"Let the function FUNCTION-NAME: DOMAIN $\to$ CODOMAIN be given by
FUNCTION-NAME(x) = ... for every $x \in DOMAIN$".

or

"Consider the function FUNCTION-NAME: DOMAIN $\to$ CODOMAIN defined as
FUNCTION-NAME(x) = ... for every $x \in DOMAIN$".

Example: The absolute value function

**Domain**

**Codomain**

**Rule**

# Defining functions recursively

When the domain of a function is a *recursively defined set*, the rule assigning images to domain elements (outputs) can also be defined recursively.

Recall: The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

**Definition** (Of a function, recursively) A function *rnalen* that computes the length of RNA strands in $S$ is defined by:

| | | | |
|---|---|---|---|
| | | $rnalen : S$ | $\to \mathbb{Z}^+$ |
| Basis Step: | If $b \in B$ then | $rnalen(b)$ | $= 1$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then | $rnalen(sb)$ | $= 1 + rnalen(s)$ |

The domain of *rnalen* is

The codomain of *rnalen* is

Example function application:

$$rnalen(\mathtt{ACU}) =$$

*Example*: A function *basecount* that computes the number of a given base $b$ appearing in a RNA strand $s$ is defined recursively:

| | | | | |
|---|---|---|---|---|
| | | $basecount : S \times B$ | $\to \mathbb{N}$ | |
| Basis Step: | If $b_1 \in B, b_2 \in B$ | $basecount(\ (b_1, b_2)\ )$ | $= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ | |
| Recursive Step: | If $s \in S, b_1 \in B, b_2 \in B$ | $basecount(\ (sb_1, b_2)\ )$ | $= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | |